

Many alerts. One incident.

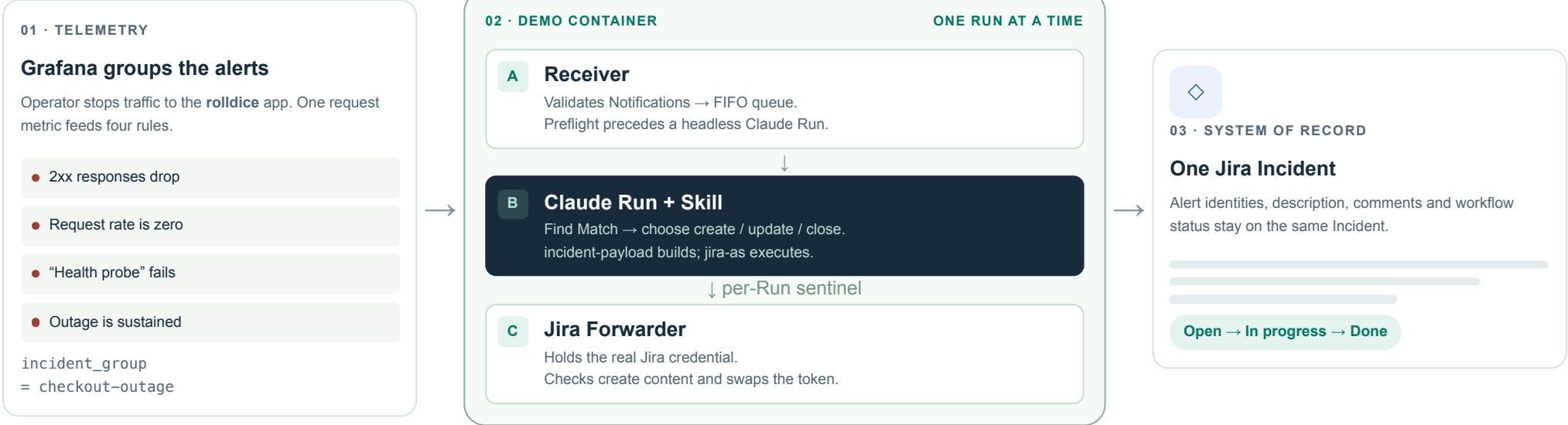
How the MVP turns a configured group of Grafana alerts into one evolving Jira Incident, with controlled AI execution and a clear path to stronger evidence.

01

Follow the alert to the Incident

THE RUNNING MVP

Docker Compose · four rules · one serialized workflow



Watch one Incident evolve

The open Match uses project, Incident type, group and session. Each new Alert adds a fingerprint; later Notifications add comments to the same Incident.

CLICK THROUGH THE LIFECYCLE

Illustrative state · recorded demo timings, not a service-level promise

One incident through the cycle

Stage	Action on the same Incident	Resulting state	Fingerprints
Create ~75 s	No open Match. Create one Incident with a description, group + session labels, two fingerprints and an opening comment.	Open	2
Related alert ~135 s	The "health probe" joins; it reads the same request metric, so it is not an independent check. Add its fingerprint and a New / Repeat comment. Move Open to Work in progress.	In progress	3
Sustained ~195 s	The sustained-outage Alert joins. Add its fingerprint and another update to the same Incident.	In progress	4
Repeat ~375 s · optional	No new fingerprints. Record a repeat comment on the same Incident. An unchanged repeat still consumes a Run.	In progress	4
Recover After traffic restarts	All Alerts in the current Notification are resolved. Comment and complete the Incident; read back status and resolution.	Completed	4

The approximate 75, 135, 195 and 375-second timings refer to Notification arrival after traffic stops, not completed Jira actions. Jira changes follow queueing and Run execution. The repeat stage is optional.

Owner-reported rehearsal, 2 October 2026: Restart traffic when the sustained-outage update Run finishes. In the recorded Sonnet presenter take, this cue occurred 209 seconds after the stop, so the optional repeat stage did not occur.

Completion applies to managed workflow states; other human-owned statuses are left to the human. Fingerprints record Alerts ever seen, rather than durable current firing state. Recovery time includes Grafana notification, queue and Run time.

Why one Incident? Shared labels identify the group; the session isolates a take; fingerprint labels identify Alerts; serial processing reduces create races. The MVP follows this explicit identity contract.

Separate judgment from mechanics

The model selects actions within the prescribed workflow. Deterministic code handles the fragile payload work. The operator and responder retain service ownership.

WHO DECIDES AND WHO ENFORCES

Current MVP boundary



DETERMINISTIC CODE

Make mechanics reliable

Grafana groups by the explicit label; the Receiver serializes Runs.

incident-payload builds JSON, labels and comment text.

The Forwarder admits one create attempt with the expected summary, description and label set.



CLAUDE RUN

Select the next action

Read the Skill and search for the prescribed open Match.

Choose create, update, close or skip; execute generated commands.

When enabled, choose read-only telemetry queries and state observations, interpretation and unknowns.



OPERATOR AND RESPONDER

Own the service

Configure grouping, Jira workflow, credentials and demo budget.

Inject the fault and restore traffic.

Inspect any evidence, decide the cause and decide remediation.

Contained execution

Non-root container, read-only root, dropped capabilities, resource limits and a narrow Claude tool allowlist. Run scratch and queue are ephemeral.

Credential mediation has a specific scope

The Run is passed a Jira sentinel. It directly holds the model credential and optional Grafana token. Jira account permissions still bound broader calls; this Forwarder is not a complete authorization policy.

Grafana queries bypass the Jira Forwarder. This stack allows anonymous Admin: wrong or missing tokens still permit queries. A valid Viewer token supplies the Run's identity; it does not enforce an access boundary.

The model chooses queries and writes Observation, Interpretation and Unknown / next check. incident-payload builds the evidence comment mechanically from the evidence file.

One-at-a-time execution and create checks support the demonstrated behavior. They do not establish durable exactly-once processing or a production security boundary.

Understand what the demo establishes

Recorded lifecycle success and the owner-reported personal-site rehearsal are the foundation. Later fixes, other telemetry backends and the future operating model each need evidence appropriate to their scope.

READ THE EVIDENCE IN LAYERS

Repository snapshot · 2 Oct 2026

● Recorded live proof

01

The incident lifecycle works

- Real Jira + real Claude replay and live traffic-stop runs reached VERIFIED.
- Three model rehearsals kept one Incident per cycle.
- Content quality differed; lifecycle success alone missed a placeholder description.

● Owner-reported rehearsal

02

Investigation in a personal-site take

- The 2 October 2026 rehearsal reports a completed Sonnet 5.5 lifecycle with investigation.
- Create-Run latency and cost were observed across takes.
- This rehearsal is not formal investigation acceptance: the pinned-image probe and a checked, faithful evidence comment on the same real Incident are still open.

● Outside this MVP

03

Broader operating model

- Evidence-based matching across a distributed cascade.
- Durable recovery, reviewed memory and Confluence learning.
- Kubernetes deployment, stronger service mediation and production scale.

OBSERVED LIFECYCLE, LATENCY AND COST

Owner-reported personal-site rehearsal, 2 October 2026: Sonnet 5.5 received the first Notification at 75 seconds with two Alerts, then updates at 134 and 196 seconds. Traffic restarted on the cue at 209 seconds; the Incident reached Completed with resolution at 291 seconds. Four Runs cost \$0.94. Across takes, investigation added approximately 2–25 seconds to the create Run: 31.7–54.4 seconds versus a 29.3-second baseline. These observations do not establish acceptance of every later fix, every telemetry backend or the unavailable-evidence fallback.

Timing components vary and the first Run may have no queue or settling delay. Optional investigation adds work. No measured responder time savings or autonomous root-cause discovery is established.

What to validate next

LEADERSHIP QUESTION	RECOMMENDED NEXT EVIDENCE
Can a cheaper model be trusted?	Replay identical fixtures after the fixes. Compare final Incident content, failed commands, elapsed time and cost.
Does investigation help responders?	Validate faithful evidence on the same real Incident, test unavailable telemetry, and measure added latency, queue delay and cost.
What blocks wider adoption?	Explicitly accept durable recovery, throughput and access boundaries. The in-memory queue and single worker trade simplicity for resilience and scale.

- [01 · MVP behavior and verification contract ↗](#)
- [03 · Earlier rehearsal evidence and post-fix gaps · 1 October 2026 ↗](#)
- [05 · Run credentials and Jira permission caveats ↗](#)
- [07 · Skill matching and workflow behavior ↗](#)

- [02 · Current runbook and investigation limits ↗](#)
- [04 · Actual queue and Run scheduling ↗](#)
- [06 · Container runtime and network setup ↗](#)
- [08 · Chapter two scope and status ↗](#)

Grounded in repository revision f910a5b, reviewed 2 October 2026. The Run preflight was added later, on main at b576463; an owner-reported replay take ran that revision the same day. This explainer simulates no live service and makes no external API calls. The 2 October lifecycle, latency and cost figures come from owner-reported personal-site rehearsal facts, separate from the linked 1 October findings record. Timings describe the demo, not a benchmark. Recommended evidence is editorial guidance.